



SHREE DEVI INSTITUTE OF TECHNOLOGY
(Approved by AICTE New Delhi, Affiliated to VTU, Belgaum)

Department of Computer Science and Engineering
Airport Road, Kenjar, Mangaluru

B.E Computer Science & Engineering

LABORATORY MANUAL

Devops LABORATORY

(BCSL657D)

LAB IN-CHARGE:

Ms. Ananya J, Ms. Dhrithi Salian

<u>Sl no</u>	<u>PROGRAMS</u>	<u>PAGE NO</u>
1	Introduction to Maven and Gradle: Overview of Build Automation Tools, Key Differences Between Maven and Gradle, Installation and Setup.	
2	Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins.	
3	Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation.	
4	Practical Exercise: Build and Run a Java Application with Maven, Migrate the Same Application to Gradle	
5	Introduction to Jenkins: What is Jenkins? Installing Jenkins on Local or Cloud Environment, Configuring Jenkins for First Use.	
6	Continuous Integration with Jenkins: Setting Up a CI Pipeline, Integrating Jenkins with Maven/Gradle, Running Automated Builds and Tests	
7	Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks, and Modules, Automating Server Configurations with Playbooks, Hands-On: Writing and Running a Basic Playbook	
8	Practical Exercise: Set Up a Jenkins CI Pipeline for a Maven Project, Use Ansible to Deploy Artifacts Generated by Jenkins.	

Appendix

Pre-requisites and Common Setup

Before starting any experiment, ensure you have the following installed and configured on your

Ubuntu system:

1. Java Development Kit (JDK):

Install OpenJDK (version 11 or above):

```
sudo apt update
```

```
sudo apt install openjdk-11-jdk
```

Verify installation:

```
java -version
```

2. Git (for version control):

```
sudo apt install git
```

Configure Git (replace with your details):

```
git config --global user.name "Your Name"
```

```
git config --global user.email "your.email@example.com"
```

3. Maven:

```
sudo apt install maven
```

Verify installation:

```
mvn -version
```

4. Gradle:

```
sudo apt install gradle
```

Verify installation:

```
gradle -v
```

5. Jenkins:**Follow these steps to install Jenkins:**

```
wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key |  
sudo apt-key add -  
  
sudo sh -c 'echo deb https://pkg.jenkins.io/debian-stable binary/ >  
/etc/apt/sources.list.d/jenkins.list'  
  
sudo apt update  
  
sudo apt install jenkins  
  
sudo systemctl start jenkins  
  
sudo systemctl status jenkins  
  
Open Jenkins in your browser at http://localhost:8080.
```

6. Ansible:

```
sudo apt update  
  
sudo apt install ansible
```

Verify installation:

```
ansible --version
```

7. Azure DevOps Account:

Create an account at Azure DevOps and set up your first project. (Steps for Azure DevOps will be covered in later experiments.)

Experiment 1: Introduction to Maven and Gradle: Overview of Build Automation Tools, Key Differences Between Maven and Gradle, Installation and Setup

1. Understanding DevOps

What is DevOps?

DevOps is a transformative culture and practice that unites software development (Dev) and IT operations (Ops) teams. By fostering collaboration and leveraging automation technologies, DevOps enables faster, more reliable code deployment to production in an efficient and repeatable manner.

(In Simple Words)

DevOps is a way of working where **software development (Dev)** and **IT operations (Ops)** teams work **together** to make software development **faster, more reliable, and efficient**.

It focuses on:

- ✓ **Teamwork:** Developers and operations teams work as one team instead of separately.
- ✓ **Automation:** Tasks like testing and deployment are done automatically, reducing manual work.
- ✓ **Faster Updates:** New features and bug fixes are released quickly and smoothly using **Continuous Integration & Continuous Delivery (CI/CD)**.
- ✓ **Monitoring & Improvement:** Systems are continuously checked, and issues are fixed before they become big problems.

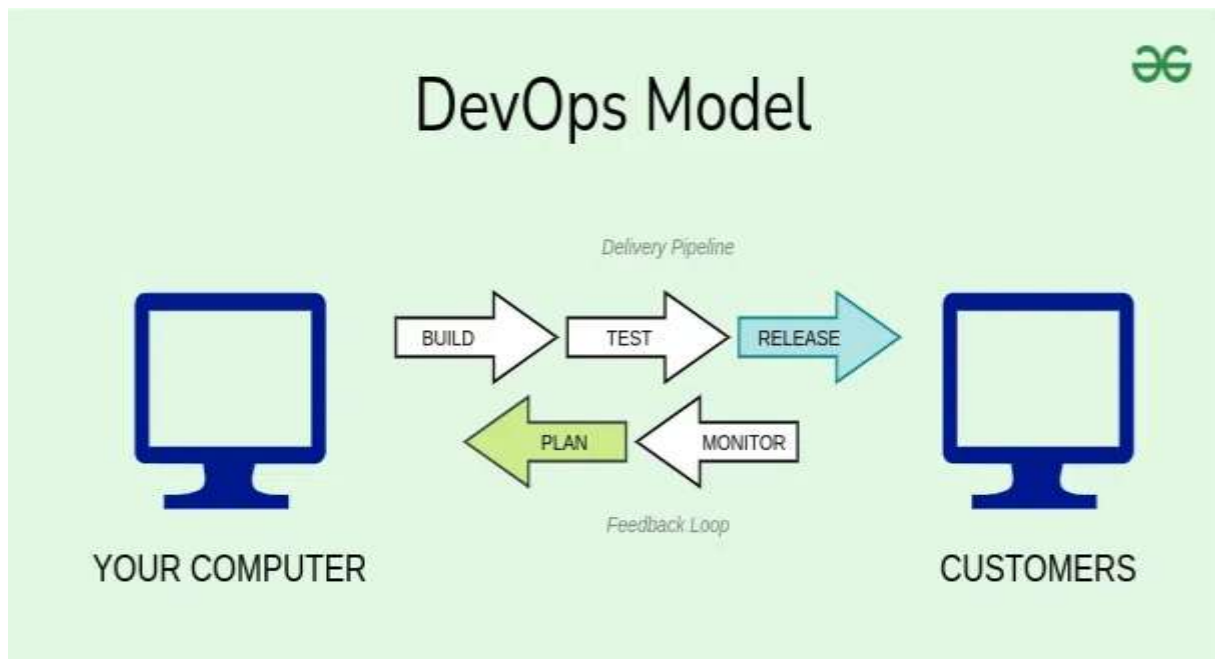
Why is DevOps Used?

DevOps is used because it makes software development faster, more reliable, and efficient. Here's why companies adopt DevOps:

- ✓ **Speed & Agility** – Software is developed and released faster, helping businesses stay ahead.
- ✓ **Better Quality** – Automated testing catches errors early, so fewer bugs reach users.
- ✓ **More Reliability** – Frequent, smaller updates prevent major system failures.

- ✓ Higher Efficiency – Automation saves time by reducing manual work and errors.
- ✓ Easy to Scale – As a company grows, DevOps ensures systems can handle more users and data.

In short: DevOps helps businesses deliver software quickly, smoothly, and with fewer problems!



This image represents the **DevOps Model**, showing how software is developed and delivered to customers in a continuous process.

1. **Your Computer (Developer's Side)** – This is where the software is created.

2. **Delivery Pipeline (Build → Test → Release)**

- **Build:** Developers write code and create the software.
- **Test:** The software is tested automatically to check for bugs.
- **Release:** Once tested, the software is deployed (made available to users).

3. **Customer (User Side)** – The final software is delivered to the customers.

4. **Feedback Loop (Plan → Monitor)**

- **Monitor:** The system tracks performance and collects feedback.
- **Plan:** Developers use the feedback to improve the software, restarting the cycle.

2. Introduction to Maven and Gradle

What is Maven?

Maven is a tool that helps developers build and manage Java projects easily. It automates tasks like compiling code, testing, and adding necessary libraries.

How It Works:

Maven uses a special file called POM (Project Object Model), written in XML, to define:

- ✓ Project structure (folders and files)
- ✓ Dependencies (external libraries needed)
- ✓ Build process (steps like compiling, testing, and packaging)

Key Features of Maven:

- Standard Structure – Follows a fixed directory format, so developers don't need to configure everything manually.
- Dependency Management – Automatically downloads required libraries from online repositories like Maven Central.
- Build Lifecycle – Handles steps like compiling, testing, packaging, and deploying in a structured way.
- Plugins – Extends functionality (e.g., running tests, generating reports, checking code quality).

What is Gradle?

Gradle is a powerful and flexible tool that automates the process of building, testing, and managing software projects. It is mainly used for Java, Kotlin, and Android development but supports many other languages too.

How It's Different from Maven?

Maven uses XML (which is strict and less flexible).

Gradle uses Groovy or Kotlin (which allows more logic, making builds more customizable

Key Features of Gradle:

- Flexible Build Scripts – Uses Groovy/Kotlin instead of rigid XML, making it more dynamic.
- Faster Builds – Only rebuilds changed files instead of everything (Incremental Builds).
- Multi-Project Support – Can handle large applications with many modules easily.
- Extensible with Plugins – Works with different languages, frameworks, and tools.
- Parallel Execution – Runs multiple tasks at the same time to speed up builds.

4. Installing Maven and Gradle on Ubuntu**A. Installing Maven****Step 1: Update Your System**

Open a terminal and run:

```
sudo apt update
```

```
sudo apt upgrade
```

Step 2: Install Java (if not already installed)

Maven requires Java. Install OpenJDK (for example, version 11):

```
sudo apt install openjdk-11-jdk
```

```
java -version
```

Expected Output (example):

```
ubuntu@ubuntu-VMware-Virtual-Platform:~$ java -version
openjdk version "11.0.26" 2025-01-21
OpenJDK Runtime Environment (build 11.0.26+4-post-Ubuntu-1ubuntu124.10)
OpenJDK 64-Bit Server VM (build 11.0.26+4-post-Ubuntu-1ubuntu124.10, mixed mode, sharing)
ubuntu@ubuntu-VMware-Virtual-Platform:~$
```

Step 3: Install Maven

Run the following command:

```
sudo apt install maven
```

After installation, check the Maven version:

```
mvn -version
```

Expected Output (example):

```
ubuntu@ubuntu-VMware-Virtual-Platform:~$ mvn -v
Apache Maven 3.8.8
Maven home: /usr/share/maven
Java version: 11.0.26, vendor: Ubuntu, runtime: /usr/lib/jvm/java-11-openjdk-amd64
Default locale: en_US, platform encoding: UTF-8
OS name: "linux", version: "6.11.0-18-generic", arch: "amd64", family: "unix"
ubuntu@ubuntu-VMware-Virtual-Platform:~$
```

B. Installing Gradle

Gradle can be installed in two primary ways: via the Ubuntu repositories (which may not be the latest version) or by manually installing the latest version.

Option 1: Install from Ubuntu Repositories

```
sudo apt install gradle
```

```
gradle -v
```

Screenshot Tip: Capture the terminal output of `gradle -v`.

Option 2: Install the Latest Version Manually (Recommended)

1. Download Gradle:

```
wget https://services.gradle.org/distributions/gradle-8.0-bin.zip
```

2. Unzip the Downloaded File:

```
Sudo unzip gradle-8.0-bin.zip -d /opt/
```

```
sudo mv gradle-8.0 /opt/gradle
```

```
ls /opt/gradle/bin
```

```
export PATH=/opt/gradle/bin:$PATH
```

4. Verify the Installation:

```
gradle -v
```

Expected Output (example):



```
ubuntu@ubuntu-VMware-Virtual-Platform:~$ gradle -v

Welcome to Gradle 8.0!

For more details see https://docs.gradle.org/8.0/release-notes.html

-----
Gradle 8.0
-----

Build time:   2023-02-13 13:15:21 UTC
Revision:     62ab9b7c7f884426cf79fbedcf07658b2dbe9e97

Kotlin:       1.8.10
Groovy:       3.0.13
Ant:          Apache Ant(TM) version 1.10.11 compiled on July 10 2021
JVM:          11.0.26 (Ubuntu 11.0.26+4-post-Ubuntu-1ubuntu124.10)
OS:           Linux 6.11.0-18-generic amd64
```

Experiment 2: Working with Maven: Creating a Maven Project, Understanding the POM File, Dependency Management and Plugins.

1. Introduction to Maven

Maven is a powerful build automation and project management tool primarily used for Java projects.

It simplifies the build process by:

- Enforcing a standard project structure (convention over configuration).
- Managing dependencies automatically by downloading them from remote repositories (e.g., Maven Central).
- Defining a clear build lifecycle (compile, test, package, install, deploy).
- Allowing the integration of various plugins to extend functionality (e.g., testing, reporting).

2. Creating a Maven Project

Step-by-Step Process

Step 1: Open Your Terminal

```
mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp  
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

- **groupId:** Uniquely identifies your project's group (like a package name).
- **artifactId:** The name of your project (the resulting artifact).
- **maven-archetype-quickstart:** A simple archetype that sets up a basic Java project with a sample unit test.
- **-DinteractiveMode=false:** Runs the command in non-interactive mode, using the provided parameters.

```
Downloading from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml
Downloaded from central: https://repo.maven.apache.org/maven2/archetype-catalog.xml (16 MB at 536 kB/s)
Downloading from central: https://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-quickstart/1.0/maven-archetype-quickstart-1.0.jar
Downloaded from central: https://repo.maven.apache.org/maven2/org/apache/maven/archetypes/maven-archetype-quickstart/1.0/maven-archetype-quickstart-1.0.jar (4.3 kB at 12 kB/s)
[INFO] -----
[INFO] Using following parameters for creating project from Old (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO] -----
[INFO] Parameter: basedir, Value: /home/ubuntu
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: MyMavenApp
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from Old (1.x) Archetype in dir: /home/ubuntu/MyMavenApp
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 01:41 min
[INFO] Finished at: 2025-02-25T21:42:16+05:30
[INFO] -----
```

Step 3: Navigate to Your Project Directory

Once the command completes successfully, change your directory to the newly created project:

- `cd MyMavenApp`

```
ubuntu@ubuntu-VMware-Virtual-Platform:~$ ls
Desktop Documents Downloads Music MyMavenApp Pictures Public snap Templates Videos
ubuntu@ubuntu-VMware-Virtual-Platform:~$ cd MyMavenApp
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp$
```

3. Maven Project Layout and Components

After generating the project, you will notice the following standard Maven directory structure:

```
MyMavenApp/  
├── pom.xml  
└── src  
    ├── main  
    │   ├── java  
    │   │   ├── com  
    │   │   │   ├── example  
    │   │   │   └── App.java  
    └── test  
        ├── java  
        │   ├── com  
        │   │   └── example  
        │   └── AppTest.java
```

```
ubuntu@ubuntu-VMware-Virtual-Platform:~$ cd MyMavenApp  
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp$ ls  
pom.xml  src  
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp$ cd src  
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src$ ls  
main  test  
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src$ cd main  
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/main$ ls  
java  
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/main$ cd java  
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/main/java$ ls  
com  
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/main/java$ cd com  
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/main/java/com$ ls  
example  
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/main/java/com$ cd example  
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/main/java/com/example$ ls  
App.java  
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/main/java/com/example$ cd ../  
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/main/java/com$ cd ../../..  
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src$ ls  
main  test
```

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src$ ls
main test
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src$ cd test
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/test$ ls
java
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/test$ cd java/com/example
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/test/java/com/example$ ls
AppTest.java
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/test/java/com/example$
```

Explanation of Key Components:

- **pom.xml:**

This is the main file in a Maven project. It has all the important details about the project, like its name, version, and the extra tools (dependencies and plugins) the project needs to run and build properly.

- **src/main/java:**

This directory holds the source code of your application. In our example, the package structure com.example is created, and you have an App.java file.

- **src/test/java:**

This folder is for writing tests to check if your main code works correctly. For example, there might be a file AppTest.java that tests the code in App.java.

Think of it like organizing your schoolwork:

- **pom.xml** is like the instruction sheet.
- **src/main/java** is where you write your main answers.
- **src/test/java** is where you double-check your answers to make sure they're correct.

4. Understanding the POM File (pom.xml)

The pom.xml file is written in XML and is essential to how Maven operates. It includes several key sections:

ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp\$ nano pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/maven-v4_0_0.xsd">
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>MyMavenApp</artifactId>
  <packaging>jar</packaging>
  <version>1.0-SNAPSHOT</version>
  <name>MyMavenApp</name>
  <url>http://maven.apache.org</url>
  <dependencies>
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>3.8.1</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

You should replace your current pom.xml with the one I provided. Your current pom.xml is missing essential elements like the build plugins, which are important for compilation and execution.

>>> nano pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <!-- Project Coordinates -->

  <groupId>com.example</groupId>

  <artifactId>MyMavenApp</artifactId>
```

```
<version>1.0-SNAPSHOT</version>
```

```
<!-- Dependencies -->
```

```
<dependencies>
```

```
  <!-- Example: JUnit for testing -->
```

```
  <dependency>
```

```
    <groupId>junit</groupId>
```

```
    <artifactId>junit</artifactId>
```

```
    <version>4.13.2</version>
```

```
    <scope>test</scope>
```

```
  </dependency>
```

```
</dependencies>
```

```
<!-- Build Configuration -->
```

```
<build>
```

```
  <plugins>
```

```
    <!-- Maven Compiler Plugin -->
```

```
    <plugin>
```

```
      <groupId>org.apache.maven.plugins</groupId>
```

```
      <artifactId>maven-compiler-plugin</artifactId>
```

```
      <version>3.8.1</version>
```

```
      <configuration>
```

```
        <source>11</source>
```

```
        <target>11</target>
```

```
      </configuration>
```

```
    </plugin>
```

```
    <!-- Maven Surefire Plugin (for running tests) -->
```

```
    <plugin>
```

```
      <groupId>org.apache.maven.plugins</groupId>
```

```
<artifactId>maven-surefire-plugin</artifactId>

<version>2.22.2</version>

</plugin>

</plugins>

</build>

</project>
```



```
GNU nano 8.1 pom.xml
<?xml version="1.0" encoding="UTF-8" ?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <!-- Project Coordinates -->
  <groupId>com.example</groupId>
  <artifactId>MyMavenApp</artifactId>
  <version>1.0-SNAPSHOT</version>

  <!-- Dependencies -->
  <dependencies>
    <!-- Example: JUnit for testing -->
    <dependency>
      <groupId>junit</groupId>
      <artifactId>junit</artifactId>
      <version>4.13.2</version>
      <scope>test</scope>
    </dependency>
  </dependencies>
</project>
```

Important Sections:

- Project Coordinates: Defines groupId, artifactId, and version.
- Dependencies: Specifies required libraries (e.g., JUnit for testing).
- Build Plugins:
 - maven-compiler-plugin: Compiles Java code.
 - maven-surefire-plugin: Runs unit tests.

Press CTRL+X, then Y, and hit Enter to save changes in nano.

5. Common Maven Commands

1. Compile the Project

mvn compile

This compiles the Java code in src/main/java.

```
ubuntu@ubuntu-VMware-Virtual-Platform:~$ cd MyMavenApp
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp$ mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:MyMavenApp >-----
[INFO] Building MyMavenApp 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ MyMavenApp ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/ubuntu/MyMavenApp/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ MyMavenApp ---
[INFO] Nothing to compile - all classes are up to date
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 2.409 s
[INFO] Finished at: 2025-03-05T22:22:36+05:30
[INFO]
```

2. Run Unit Tests

mvn test

This runs the test cases in src/test/java.

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp$ mvn test
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/ubuntu/MyMavenApp/target/test-classes
[INFO]
[INFO] --- maven-surefire-plugin:2.22.2:test (default-test) @ MyMavenApp ---
[INFO]
[INFO] T E S T S
[INFO]
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.118 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 3.304 s
[INFO] Finished at: 2025-03-05T23:12:41+05:30
[INFO]
```

3. Package the Application

mvn package

This command:

- Compiles the code.
- Runs the tests.
- Creates a JAR file inside the target/ directory.

```
INFO] --- maven-surefire-plugin:2.22.2:test (default-test) @ MyMavenApp ---
INFO]
INFO] -----
INFO] T E S T S
INFO] -----
INFO] Running com.example.AppTest
INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.02 s - in com.example.AppTest
INFO]
INFO] Results:
INFO]
INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
INFO]
INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ MyMavenApp ---
INFO] Building jar: /home/ubuntu/MyMavenApp/target/MyMavenApp-1.0-SNAPSHOT.jar
INFO]
INFO] -----
INFO] BUILD SUCCESS
INFO]
INFO] -----
INFO] Total time: 2.760 s
INFO] Finished at: 2025-03-05T23:12:53+05:30
INFO]
INFO] -----
ubuntu@ubuntu:~/Virtual-Platform:~/MyMavenApp$ ls target/
```

4. Check the JAR file by running:

- **ls target/**

This command compiles, tests, and packages your code into a JAR file located in the target directory.

```
ubuntu@ubuntu:~/Virtual-Platform:~/MyMavenApp$ ls target/
classes          generated-test-sources  naven-status          surefire-reports
generated-sources  naven-archiver         MyMavenApp-1.0-SNAPSHOT.jar  test-classes
```

5. Clean the Project:

- **mvn clean**

This removes any files generated by previous builds.

Deletes generated files from the target/ directory.

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp$ mvn clean
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:MyMavenApp >-----
[INFO] Building MyMavenApp 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-clean-plugin:2.5:clean (default-clean) @ MyMavenApp ---
[INFO] Deleting /home/ubuntu/MyMavenApp/target
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 0.408 s
[INFO] Finished at: 2025-03-05T22:31:46+05:30
[INFO]
[INFO] -----
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp$ cd src/main/
```

6. Running the Application

To run the Java application, execute:

- mvn package
- ls target/
- java -cp target/MyMavenApp-1.0-SNAPSHOT.jar com.example.App

```
Mar 5 22:18
ubuntu@ubuntu-VMware-Virtual-Platform: ~/MyMavenApp/src/main/java/com/example
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running com.example.AppTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.077 s - in com.example.AppTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO]
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ MyMavenApp ---
[INFO] Building jar: /home/ubuntu/MyMavenApp/target/MyMavenApp-1.0-SNAPSHOT.jar
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 4.348 s
[INFO] Finished at: 2025-03-05T22:31:15+05:30
[INFO]
[INFO] -----
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp$ ls target/
classes generated-test-sources maven-status surefire-reports
generated-sources maven-archiver MyMavenApp-1.0-SNAPSHOT.jar test-classes
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp$ cd java/com/example
bash: cd: java/com/example: No such file or directory
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp$ java -cp target/MyMavenApp-1.0-SNAPSHOT.jar com.example.App
Hello World!
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp$ mvn clean
```

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp$ cd src/main/
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/main$ cd java/com/example
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/main/java/com/example$ ls
app.java  App.java
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/main/java/com/example$ nano App.java
ubuntu@ubuntu-VMware-Virtual-Platform:~/MyMavenApp/src/main/java/com/example$
```

```
GNU nano 8.1 App.java
package com.example;

/**
 * Hello world!
 */
public class App
{
    public static void main( String[] args )
    {
        System.out.println( "Hello World!" );
    }
}
```

Final Summary of Commands:

Command Purpose:

- **mvn -version** - Check if Maven is installed.
- **mvn archetype:generate -DgroupId=com.example -DartifactId=MyMavenApp -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false** - Create a Maven project.
- **cd MyMavenApp** - Navigate into the project directory.
- **nano pom.xml** - Open and edit pom.xml.
- **mvn compile** - Compile the Java project.
- **mvn test** - Run test cases.
- **mvn package** - Create a JAR file in target/.
- **ls target/** - List files in target/ directory.
- **mvn clean** - Remove compiled files and dependencies.
- **java -cp target/MyMavenApp-1.0-SNAPSHOT.jar com.example.App** - Run the application.

Experiment 3: Working with Gradle: Setting Up a Gradle Project, Understanding Build Scripts (Groovy and Kotlin DSL), Dependency Management and Task Automation.

1. Introduction to Gradle:

Gradle is a modern build automation tool designed to be highly flexible, fast, and scalable. It is widely used in Java projects, Android development, and many multi-language projects.

Step 1: Create a New Gradle Project

1. Create a new directory for your project and navigate to it:

1. mkdir Hellogradle
2. cd Hellogradle

2. Run the Gradle init command:

3. gradle init --type java-application

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloGradle$ gradle init --type java-application

Select build script DSL:
  1: Groovy
  2: Kotlin
Enter selection (default: Groovy) [1..2] 1

Generate build using new APIs and behavior (some features may change in the next minor release)? (default: no) [yes, no] yes

Select test framework:
  1: JUnit 4
  2: TestNG
  3: Spock
  4: JUnit Jupiter
Enter selection (default: JUnit Jupiter) [1..4] 1,2,3,4
Please enter a value between 1 and 4: 1 and 4
Please enter a value between 1 and 4: 1 4
Please enter a value between 1 and 4: 1

Project name (default: HelloGradle):
Source package (default: hellogradle):

> Task :init
Get more help with your project: https://docs.gradle.org/8.0/samples/sample_building_java_applications.html

BUILD SUCCESSFUL in 50s
2 actionable tasks: 2 executed
```

Step 2: Explore the Project Structure

After project creation, the directory structure typically looks like this:

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/Hellogradle$ tree
.
├── build.gradle
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
├── settings.gradle
└── src
    ├── main
    │   └── java
    │       └── App.java
    └── test
        └── java
            └── AppTest.java

8 directories, 8 files
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/Hellogradle$
```

Explanation of Components:

- build.gradle:

This is the main build script written in Groovy (or Kotlin if you choose). It defines plugins, repositories, dependencies, and tasks.

- settings.gradle:

A small script that defines the project's name and, in multi-project builds, the included subprojects.

- gradlew / gradlew.bat:

The Gradle wrapper scripts. They allow you to run Gradle without requiring a separate installation on every machine by automatically downloading the correct Gradle version.

- `src/main/java`:

Contains your application's source code.

- `src/test/java`:

Contains your unit tests.

3. Understanding Gradle Build Scripts

A build script in Gradle is a programmatic file that instructs Gradle on how to build your project. It can be written in two main DSLs:

- Groovy DSL (`build.gradle`): The traditional and most common syntax.
- Kotlin DSL (`build.gradle.kts`): A statically-typed alternative that leverages Kotlin's language features.

A. Groovy DSL Example (`build.gradle`)

Below is an example of a basic `build.gradle` for a Java application using Groovy DSL:

➤ **gedit `build.gradle`**

```
plugins {  
    // Apply the Java plugin for compiling Java code  
    id 'java'  
    // Apply the application plugin to add support for building an application  
    id 'application'  
}  
  
group = 'com.example'  
  
version = '1.0'  
  
repositories {  
    // Use Maven Central for resolving dependencies.
```

```
mavenCentral()

}

dependencies {

    // Define your dependencies. For example, JUnit for testing:

    testImplementation 'junit:junit:4.13.2'

}

application {

    // Define the main class for the application.

    mainClass = 'com.example.App'

}

// A custom task example: printing a greeting

task hello {

    doLast {

        println 'Hello, Gradle!'

    }

}
```

A screenshot of a code editor window titled 'build.gradle' with the path '~/devops/helloGradle'. The editor contains a Gradle build script. The script defines two plugins: 'java' and 'application'. It sets the group to 'com.example' and the version to '1.0'. It adds Maven Central as a repository. It defines dependencies for JUnit. It sets the main class to 'com.example.App'. Finally, it defines a custom task named 'hello' that prints 'Hello, Gradle!' when executed.

```
1 plugins {  
2 // Apply the Java plugin for compiling Java code  
3 id 'java'  
4 // Apply the application plugin to add support for building an application  
5 id 'application'  
6 }  
7 group = 'com.example'  
8 version = '1.0'  
9 repositories {  
10 // Use Maven Central for resolving dependencies.  
11 mavenCentral()  
12 }  
13 dependencies {  
14 // Define your dependencies. For example, JUnit for testing:  
15 testImplementation 'junit:junit:4.13.2'  
16 }  
17 application {  
18 // Define the main class for the application.  
19 mainClass = 'com.example.App'  
20 }  
21 // A custom task example: printing a greeting  
22 task hello {  
23 doLast {  
24 println 'Hello, Gradle!'  
25 }  
26 }
```

4. Run the Custom Task:

- Command:

➤ **gradle hello**

- Expected Output:

You should see:

> Task :hello

Hello, Gradle!

A screenshot of a terminal window. The prompt is 'ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloGradle\$'. The user has entered 'gradle hello'. The output shows the task ':hello' being executed, printing 'Hello, Gradle!'. The build is successful, taking 2 seconds, and 1 actionable task was executed.

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloGradle$ gradle hello  
  
> Task :hello  
Hello, Gradle!  
  
BUILD SUCCESSFUL in 2s  
1 actionable task: 1 executed
```

Task Dependencies:

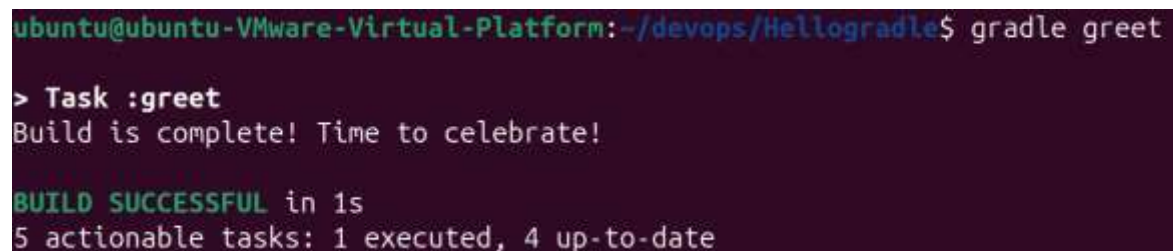
You can make one task depend on another. For instance, you might want your custom task to run after the build task:

Groovy DSL:

```
task greet(dependsOn: build) {  
  
    doLast {  
  
        println 'Build is complete! Time to celebrate!'  
  
    }  
  
}
```

Command:

➤ **gradle greet**



```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloGradle$ gradle greet  
  
> Task :greet  
Build is complete! Time to celebrate!  
  
BUILD SUCCESSFUL in 1s  
5 actionable tasks: 1 executed, 4 up-to-date
```

5. Running and Verifying Your Gradle Project

Common Gradle Commands:

Build the Project:

Compile, Test, and Package:

- Command:

gradle build

- What it does:

- o Compiles source code (compileJava).
- o Runs tests (test).
- o Packages the application into a JAR file (jar).
- Expected

Look for a "BUILD SUCCESSFUL" message.

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloGradle$ gradle build

BUILD SUCCESSFUL in 10s
11 actionable tasks: 11 executed
```

B. Kotlin DSL Example (build.gradle.kts)

Step 1: Create a New Gradle Project

1. Create a new directory for your project and navigate to it:

1. mkdir HelloGradle
2. cd HelloGradle

2. Run the Gradle init command:

3. gradle init --type java-application

```
at java.base/java.lang.Thread.run(Thread.java:829)
Stopping file watching and invalidating VFS after an error happened

Select build script DSL:
 1: Groovy
 2: Kotlin
Enter selection (default: Groovy) [1..2] 2

Generate build using new APIs and behavior (some features may change in the next minor release)? (default: no) [yes, no] yes
Select test framework:
 1: JUnit 4
 2: TestNG
 3: Spock
 4: JUnit Jupiter
Enter selection (default: JUnit Jupiter) [1..4] 4

Project name (default: HelloGradle):
Source package (default: hellogradle):

> Task :init
Get more help with your project: https://docs.gradle.org/8.0/samples/sample_building_java_applications.html

BUILD SUCCESSFUL in 17s
2 actionable tasks: 2 executed
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloGradle$
```

Step 2: Explore the Project Structure

After project creation, the directory structure typically looks like this:

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloGradle$ tree
.
├── app
│   ├── build.gradle.kts
│   └── src
│       ├── main
│       │   ├── java
│       │   │   └── hellogradle
│       │   │       └── App.java
│       │   └── resources
│       └── test
│           ├── java
│           │   └── hellogradle
│           │       └── AppTest.java
│           └── resources
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
└── settings.gradle.kts

13 directories, 8 files
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloGradle$
```

B. Kotlin DSL Example (build.gradle.kts)

Here's how the same build configuration might look in Kotlin DSL:

Create a file named build.gradle.kts (or convert your file) with the following content:Run the Custom Task:

```
plugins {

// Apply the Java plugin for compiling Java code

java
```

```
// Apply the application plugin to add support for building an application
```

```
application
```

```
}
```

```
group = "com.example"
```

```
version = "1.0"
```

```
repositories {
```

```
mavenCentral()
```

```
}
```

```
dependencies {
```

```
// Define dependencies using Kotlin DSL syntax
```

```
testImplementation("junit:junit:4.13.2")
```

```
}
```

```
application {
```

```
// Set the main class for the application
```

```
mainClass.set("com.example.App")
```

```
}
```

```
// A custom task example using Kotlin DSL
```

```
tasks.register("hello") {
```

```
doLast {
```

```
println("Hello, Gradle with Kotlin DSL!")
```

```
}
```

```
}
```

Command:

./gradlew hello

Expected Output:

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloGradle$ ./gradlew hello

> Task :hello
Hello, Gradle with Kotlin DSL!

BUILD SUCCESSFUL in 4s
1 actionable task: 1 executed
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloGradle$
```

Task Dependencies:

You can make one task depend on another. For instance, you might want your custom task to run after the build task:

Kotlin DSL:

```
tasks.register("greet") {
    dependsOn("build")

    doLast {
        println("Build is complete! Time to celebrate!")
    }
}
```

Command:

./gradlew greet

```
500/styles/fair-dark.xml: missing version attribute
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloGradle$ ./gradlew greet

> Task :greet
Build is complete! Time to celebrate!

BUILD SUCCESSFUL in 4s
5 actionable tasks: 5 executed
```

5. Running and Verifying Your Gradle Project

Common Gradle Commands:

Build the Project:

Compile, Test, and Package:

- Command:

gradle build

- What it does:

- o Compiles source code (compileJava).
- o Runs tests (test).
- o Packages the application into a JAR file (jar).

- Expected

Look for a "BUILD SUCCESSFUL" message.

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloGradle$ gradle build  
  
BUILD SUCCESSFUL in 4s  
11 actionable tasks: 7 executed, 4 up-to-date  
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloGradle$
```

Experiment 4: Practical Exercise: Build and Run a Java Application with Maven, Migrate the Same Application to Gradle

Part A: Build and Run a Java Application with Maven

1. Create the Maven Project

- 1. Open your Terminal.**
- 2. Generate the Maven Project using the Quickstart Archetype:**

Type the following command and press Enter:

```
mvn archetype:generate -DgroupId=com.example -DartifactId=HelloMaven  
-DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
```

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops$ mvn archetype:generate -DgroupId=com.example -DartifactId=HelloMaven -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
[INFO] Scanning for projects...
[INFO]
[INFO] -----< org.apache.maven.plugins:maven-archetype-quickstart:1.0:generate (default-cli) @ standalone-pom -----
[INFO] Building Maven Stub Project (No POM) 1
[INFO] -----[ pom ]-----
[INFO]
[INFO] >>> maven-archetype-plugin:3.1.1:generate (default-cli) > generate-sources @ standalone-pom >>>
[INFO]
[INFO] <<< maven-archetype-plugin:3.1.1:generate (default-cli) < generate-sources @ standalone-pom <<<
[INFO]
[INFO] --- maven-archetype-plugin:3.1.1:generate (default-cli) @ standalone-pom ---
[INFO] Generating project in Batch mode
[INFO]
[INFO] Using following parameters for creating project from DId (1.x) Archetype: maven-archetype-quickstart:1.0
[INFO]
[INFO] Parameter: basedir, Value: /home/ubuntu/devops
[INFO] Parameter: package, Value: com.example
[INFO] Parameter: groupId, Value: com.example
[INFO] Parameter: artifactId, Value: HelloMaven
[INFO] Parameter: packageName, Value: com.example
[INFO] Parameter: version, Value: 1.0-SNAPSHOT
[INFO] project created from DId (1.x) Archetype in dir: /home/ubuntu/devops/HelloMaven
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] Total time: 2.824 s
[INFO]
```

2. Change Directory into the Newly Created Project:

```
cd HelloMaven
```

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops$ ls
HelloMaven  HelloMaven  HelloMaven  MyMavenApp
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops$ cd HelloMaven
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloMaven$ ls
pom.xml  src
```

3. Understanding the POM File (pom.xml)

You should replace your current pom.xml with the one I provided. Your current pom.xml is missing essential elements like the build plugins, which are important for compilation and execution.

gedit pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>

  <artifactId>HelloMaven</artifactId>

  <version>1.0-SNAPSHOT</version>

  <dependencies>

    <dependency>

      <groupId>junit</groupId>

      <artifactId>junit</artifactId>

      <version>4.13.2</version>

      <scope>test</scope>

    </dependency>

  </dependencies>

  <build>

    <plugins>

      <plugin>

        <groupId>org.apache.maven.plugins</groupId>
```

```
<artifactId>maven-compiler-plugin</artifactId>

<version>3.8.1</version>

<configuration>

<source>11</source>

<target>11</target>

</configuration>

</plugin>

</plugins>

</build>

</project>
```

4. Build the Maven Project

- mvn compile
- mvn test
- mvn package
- ls target
- java -cp target/HelloMaven-1.0-SNAPSHOT.jar com.example.App

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloMaven$ mvn compile
[INFO] Scanning for projects...
[INFO]
[INFO] -----< com.example:HelloMaven >-----
[INFO] Building HelloMaven 1.0-SNAPSHOT
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- maven-resources-plugin:2.6:resources (default-resources) @ HelloMaven ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/ubuntu/devops/HelloMaven/src/main/resources
[INFO]
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ HelloMaven ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/ubuntu/devops/HelloMaven/target/classes
[INFO]
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 2.725 s
[INFO] Finished at: 2025-03-24T22:57:32+05:30
[INFO] -----
```

```
[INFO] --- maven-compiler-plugin:3.8.1:compile (default-compile) @ HelloMaven ---
[INFO] Nothing to compile - all classes are up to date
[INFO] --- maven-resources-plugin:2.6:testResources (default-testResources) @ HelloMaven ---
[WARNING] Using platform encoding (UTF-8 actually) to copy filtered resources, i.e. build is platform dependent!
[INFO] skip non existing resourceDirectory /home/ubuntu/devops/HelloMaven/src/test/resources
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ HelloMaven ---
[INFO] Changes detected - recompiling the module!
[WARNING] File encoding has not been set, using platform encoding UTF-8, i.e. build is platform dependent!
[INFO] Compiling 1 source file to /home/ubuntu/devops/HelloMaven/target/test-classes
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ HelloMaven ---
[INFO] Surefire report directory: /home/ubuntu/devops/HelloMaven/target/surefire-reports

-----
T E S T S
-----
Running com.example.AppTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.098 sec

Results :

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 3.714 s
[INFO] Finished at: 2025-03-24T22:57:43+05:30
[INFO] -----

[INFO] skip non existing resourceDirectory /home/ubuntu/devops/HelloMaven/src/test/resources
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ HelloMaven ---
[INFO] Nothing to compile - all classes are up to date
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ HelloMaven ---
[INFO] Surefire report directory: /home/ubuntu/devops/HelloMaven/target/surefire-reports

-----
T E S T S
-----
Running com.example.AppTest
Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.069 sec

Results :

Tests run: 1, Failures: 0, Errors: 0, Skipped: 0

[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ HelloMaven ---
[INFO] Building jar: /home/ubuntu/devops/HelloMaven/target/HelloMaven-1.0-SNAPSHOT.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 2.798 s
[INFO] Finished at: 2025-03-24T23:01:07+05:30
[INFO] -----
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloMaven$ ls target
classes generated-sources generated-test-sources HelloMaven-1.0-SNAPSHOT.jar maven-archiver maven-status surefire-reports test-classes
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloMaven$ java -cp target/HelloMaven-1.0-SNAPSHOT.jar com.example.App
Hello World!
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloMaven$
```

Part B: Migrate the Application to Gradle

1. Open a New Terminal Window or Navigate Back to Your Workspace.
2. Create a New Directory for the Gradle Project:
 1. mkdir HelloMavenGradle
 2. cd HelloMavenGradle

What it does:

Creates and navigates into a folder named HelloMavenGradle for your new

Gradle project.

3. Initialize the Gradle Project Using the Java Application Type:

1. gradle init --type java-application

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloMavenGradle$ gradle init --type java-application
Starting a Gradle Daemon, 1 incompatible and 1 stopped Daemons could not be reused, use --status for details

Select build script DSL:
  1: Groovy
  2: Kotlin
Enter selection (default: Groovy) [1..2] 1

Generate build using new APIs and behavior (some features may change in the next minor release)? (default: no) [yes, no]

Select test framework:
  1: JUnit 4
  2: TestNG
  3: Spock
  4: JUnit Jupiter
Enter selection (default: JUnit Jupiter) [1..4] 4

Project name (default: HelloMavenGradle):
Source package (default: hellonavengradle):

> Task :init
Get more help with your project: https://docs.gradle.org/8.0/samples/sample_building_java_applications.html

BUILD SUCCESSFUL in 34s
2 actionable tasks: 2 executed
```

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloMavenGradle$ tree
.
├── app
│   ├── build.gradle
│   └── src
│       ├── main
│       │   ├── java
│       │   │   └── hellonavengradle
│       │   │       └── App.java
│       │   └── resources
│       └── test
│           ├── java
│           │   ├── hellonavengradle
│           │   └── AppTest.java
│           └── resources
├── gradle
│   └── wrapper
│       ├── gradle-wrapper.jar
│       └── gradle-wrapper.properties
├── gradlew
├── gradlew.bat
└── settings.gradle

13 directories, 8 files
```

4. gedit build.gradle

```
plugins {  
    id 'application'  
    id 'java'  
}  
application {  
    mainClass = 'com.example.App' // Correct way in Gradle 7+  
}  
repositories {  
    mavenCentral()  
}  
dependencies {  
    implementation 'org.apache.commons:commons-lang3:3.12.0'  
    testImplementation 'junit:junit:4.13.2'  
}
```

5. Ensure the Package Declaration Matches:

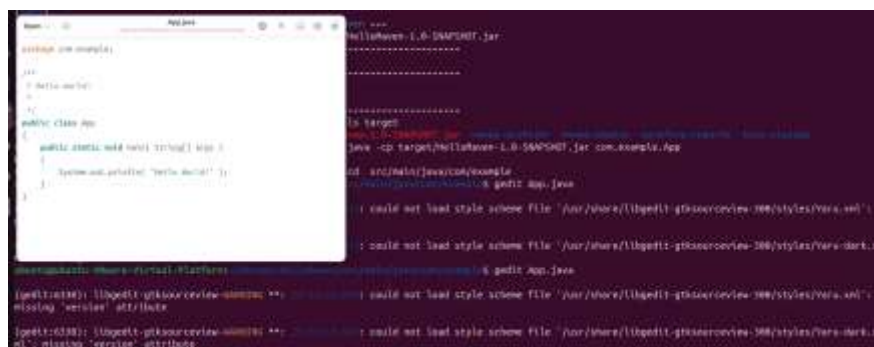
- o. In the Maven project, the class is in the package com.example.
- o. If the Gradle project's default App.java is not in this package, create the proper directory structure:

- **mkdir -p src/main/java/com/example**

- o. Move (or copy) the App.java file into the com/example folder:

- **gedit src/main/java/com/example/App.java**

- o. Copy the content from the maven project App.java file:



6. Build and Run the Gradle Application

- gradle build : Compiles the source code, runs tests, and packages the application.
- gradle run

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloMavenGradle$ gradle build

BUILD SUCCESSFUL in 3s
12 actionable tasks: 12 up-to-date
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloMavenGradle$ gradle run

> Task :run
Hello World!

> Task :app:run
Hello World!

BUILD SUCCESSFUL in 1s
4 actionable tasks: 2 executed, 2 up-to-date
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloMavenGradle$
```

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloMavenGradle$ gradle build

BUILD SUCCESSFUL in 2s
12 actionable tasks: 5 executed, 7 up-to-date
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloMavenGradle$ gradle run

> Task :run
World!

> Task :app:run
Hello World!

BUILD SUCCESSFUL in 2s
4 actionable tasks: 2 executed, 2 up-to-date
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops/HelloMavenGradle$
```

Experiment 5: Introduction to Jenkins: What is Jenkins? Installing Jenkins on Local or Cloud Environment, Configuring Jenkins for First Use.

1. What Is Jenkins?

Jenkins is a free tool that helps developers automate tasks like building, testing, and delivering software. Instead of manually checking and updating code, Jenkins does it for you. Think of it like a robot assistant for software developers. Whenever new code is added, Jenkins:

1. Builds the software (compiles the code).
2. Tests it to check for errors.
3. Deploys it (sends it to production or testing environments).

This makes sure that developers can quickly find and fix issues, leading to faster and smoother software updates.

Key Functionalities:

- **Continuous Integration/Delivery:** Automatically builds, tests, and deploys code after each commit.
- **Extensible Plugin Ecosystem:** Over 1,500 plugins allow integration with numerous tools such as Git, Maven, Gradle, Docker, and many cloud providers.
- **Automated Builds:** Jenkins can poll version control systems, trigger builds on code changes, and even schedule builds.
- **Pipeline as Code:** With the introduction of Jenkins Pipeline (using either scripted or declarative syntax), you can define your entire build process in a code file (Jenkinsfile) and store it alongside your code.
- **Distributed Builds:** Jenkins can distribute workloads across multiple machines, helping to run tests and builds faster.
- **Monitoring and Reporting:** Provides detailed logs, test reports, and build history.

Why Use Jenkins?

- ✓ **Saves Time** – No need to do things manually.
- ✓ **Faster Software Development** – Finds mistakes early.
- ✓ **Works with Any Project** – Supports many tools and programming languages.
- ✓ **Handles Big Projects** – Can work with multiple developers and computers

2. Installing Jenkins on a Local Machine (Ubuntu).

Step 1: Install Java

Jenkins requires Java to run:

java -version

```
ubuntu@ubuntu-VMware-Virtual-Platform:~$ java -version
openjdk version "11.0.26" 2025-01-21
OpenJDK Runtime Environment (build 11.0.26+4-post-Ubuntu-1ubuntu124.10)
OpenJDK 64-Bit Server VM (build 11.0.26+4-post-Ubuntu-1ubuntu124.10, mixed mode, sharing)
ubuntu@ubuntu-VMware-Virtual-Platform:~$
```

This is not supporting us to install the Jenkins so we are updating the java version from 11 to 17.

➤ sudo apt install fontconfig openjdk-17-jre

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops$ sudo apt install fontconfig openjdk-17-jre
fontconfig is already the newest version (2.15.0-1ubuntu2).
fontconfig set to manually installed.
The following packages were automatically installed and are no longer required:
  bnd                                libgradle-core-java          libnative-platform-java
  groovy                           libgradle-plugins-java      libnative-platform-jni
  ivy                              libhttpclient-java          libnekohtml-java
  java-wrappers                   libhttpcore-java            libobjenesis-java
  javascrypt-common               libjarjar-java              libosgi-annotation-java
  libbcpg-java                    libjatl-java                 libosgi-compendium-java
  libbcprov-java                  libjavaewah-java            libosgi-core-java
  libbindex-java                 libjaxen-java               libplexus-container-default-java
  libcommons-codec-java          libjcifs-java               libpolyglot-maven-java
  libcommons-collections3-java   libjcip-annotations-java    libqdox-java
  libcommons-compress-java       libjcsp-java                libreflectasm-java
  libcommons-lang-java           libjformatstring-java       libreflectasm-java-doc
  libdd-plist-java               libjgit-java                librhino-java
  libdom4j-java                  libjna-java                 libsimpl-http-java
  libeclipse-jdt-annotation-java libjna-jni                  libwagon-http-java
  libfelix-framework-java        libjs-jquery                libxbean-reflect-java
  libfelix-gogo-runtime-java     libjsoup-java               libxerces2-java
  libfelix-osgi-obr-java         libjsr166y-java             libxml-commons-external-java
  libfelix-resolver-java         libkrvn-java                libxpp3-java
```

➤ Java --version

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops$ java --version
openjdk 17.0.14 2025-01-21
OpenJDK Runtime Environment (build 17.0.14+7-Ubuntu-124.10)
OpenJDK 64-Bit Server VM (build 17.0.14+7-Ubuntu-124.10, mixed mode, sharing)
```

Step 2: Add the Jenkins Repository Key

Download and add the repository key so that your system trusts the Jenkins packages:

- `sudo wget -O /usr/share/keyrings/jenkins-keyring.asc https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key`

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops$ sudo wget -O /usr/share/keyrings/jenkins-keyring.asc https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
--2025-04-02 22:27:25-- https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key
Resolving pkg.jenkins.io (pkg.jenkins.io)... 151.101.158.133, 2a04:4e42:25::645
Connecting to pkg.jenkins.io (pkg.jenkins.io)|151.101.158.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3175 (3.1K) [application/pgp-keys]
Saving to: '/usr/share/keyrings/jenkins-keyring.asc'

/usr/share/keyrings/jenkins-keyring.asc 100%[=====] 3.10K --KB/s in 0.04s

2025-04-02 22:27:27 (80.7 KB/s) - '/usr/share/keyrings/jenkins-keyring.asc' saved [3175/3175]
```

Step 3: Add the Jenkins Repository

Add the Jenkins repository to your system's sources list:

- `echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]" https://pkg.jenkins.io/debian-stable binary/ | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null`

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops$ echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc]" https://pkg.jenkins.io/debian-stable binary/ | sudo tee /etc/apt/sources.list.d/jenkins.list > /dev/null
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops$ sudo apt-get update
Ign:1 https://pkg.jenkins.io/debian-stable binary/ InRelease
Hit:2 https://pkg.jenkins.io/debian-stable binary/ Release
Hit:3 http://security.ubuntu.com/ubuntu oracular-security InRelease
Hit:5 http://in.archive.ubuntu.com/ubuntu oracular InRelease
Hit:6 http://in.archive.ubuntu.com/ubuntu oracular-updates InRelease
Hit:7 http://in.archive.ubuntu.com/ubuntu oracular-backports InRelease
```

Step 4: Update the Repository and Install Jenkins

Update the package list and install Jenkins:

- `sudo apt update`

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops$ sudo apt-get update
Ign:1 https://pkg.jenkins.io/debian-stable binary/ InRelease
Hit:2 https://pkg.jenkins.io/debian-stable binary/ Release
Hit:3 http://security.ubuntu.com/ubuntu oracular-security InRelease
Hit:5 http://in.archive.ubuntu.com/ubuntu oracular InRelease
Hit:6 http://in.archive.ubuntu.com/ubuntu oracular-updates InRelease
Hit:7 http://in.archive.ubuntu.com/ubuntu oracular-backports InRelease
Reading package lists... Done
```

- sudo apt-get install Jenkins

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops$ sudo apt-get install jenkins
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
jenkins is already the newest version (2.492.3).
The following packages were automatically installed and are no longer required:
  bnd groovy ivy java-wrappers javascript-common libbcpg-java libbcprov-java libbindex-java
  libcommons-codec-java libcommons-collections3-java libcommons-compress-java libcommons-lang-java
  libdd-plist-java libdom4j-java libeclipse-jdt-annotation-java libfelix-framework-java
  libfelix-gogo-runtime-java libfelix-osgi-obr-java libfelix-resolver-java libfindbugs-java
  libgoogle-gson-java libgpars-groovy-java libgradle-core-java libgradle-plugins-java libhttpclient-java
  libhttpcore-java libjarjar-java libjatl-java libjavaewah-java libjaxen-java libjcifs-java
  libjcip-annotations-java libjcsp-java libjformatstring-java libjgit-java libjna-java libjna-jni
  libjs-jquery libjsoup-java libjsr166y-java libkryo-java libkxml2-java libminlog-java
  libmultiverse-core-java libnative-platform-java libnative-platform-jni libnekohtml-java
  libobjenesis-java libosgi-annotation-java libosgi-compendium-java libosgi-core-java
  libplexus-container-default-java libpolyglot-maven-java libqdox-java libreflectasm-java
  libreflectasm-java-doc librhino-java libsimple-http-java libwagon-http-java libxbean-reflect-java
  libxerces2-java libxml-commons-external-java libxpp3-java libxstream-java python3-netifaces
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 4 not upgraded.
```

Step 5: Start and Enable the Jenkins Service

Start Jenkins and configure it to start automatically on boot:

- sudo systemctl start jenkins

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops$ sudo systemctl start jenkins
Warning: The unit file, source configuration file or drop-ins of jenkins.service changed on disk. Run 'systemctl daemon-reload' to reload units.
```

- sudo systemctl daemon-reload
- sudo systemctl start jenkins
- sudo systemctl status Jenkins

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops$ sudo systemctl daemon-reload
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops$ sudo systemctl start jenkins
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops$ sudo systemctl status jenkins
● jenkins.service - Jenkins Continuous Integration Server
   Loaded: loaded (/usr/lib/systemd/system/jenkins.service; enabled; preset: enabled)
   Active: active (running) since Wed 2025-04-02 22:29:22 IST; 38s ago
 Invocation: e42de59cdaca46af98faae924a85805c
    Main PID: 7745 (java)
      Tasks: 50 (limit: 3935)
     Memory: 634.4M (peak: 644.2M)
        CPU: 33.547s
    CGroup: /system.slice/jenkins.service
            └─7745 /usr/bin/java -Djava.awt.headless=true -jar /usr/share/java/jenkins.war --webroot=/var

Apr 02 22:28:32 ubuntu-VMware-Virtual-Platform jenkins[7745]: d84680b78b434e5fa5f0877ce5a0ce58
Apr 02 22:28:32 ubuntu-VMware-Virtual-Platform jenkins[7745]: This may also be found at: /var/lib/jenkins/
Apr 02 22:28:32 ubuntu-VMware-Virtual-Platform jenkins[7745]: *****
Apr 02 22:28:32 ubuntu-VMware-Virtual-Platform jenkins[7745]: *****
Apr 02 22:28:32 ubuntu-VMware-Virtual-Platform jenkins[7745]: *****
Apr 02 22:29:20 ubuntu-VMware-Virtual-Platform jenkins[7745]: 2025-04-02 16:59:20.837+0000 [id=48]
Apr 02 22:29:20 ubuntu-VMware-Virtual-Platform jenkins[7745]: 2025-04-02 16:59:20.838+0000 [id=48]
Apr 02 22:29:22 ubuntu-VMware-Virtual-Platform jenkins[7745]: 2025-04-02 16:59:22.052+0000 [id=33]
Apr 02 22:29:22 ubuntu-VMware-Virtual-Platform jenkins[7745]: 2025-04-02 16:59:22.079+0000 [id=23]
```

What it does:

- start jenkins launches the Jenkins service.
- status jenkins confirms Jenkins is running.

Step 6: Access Jenkins on the Local Machine

1. Open web browser.
2. Navigate to: <http://localhost:8080>
3. Unlock Jenkins: The initial Jenkins screen will ask for an administrator password.

Retrieve it by running:

4. `sudo more /var/lib/jenkins/secrets/initialAdminPassword`
 - What it does: Displays the auto-generated admin password.

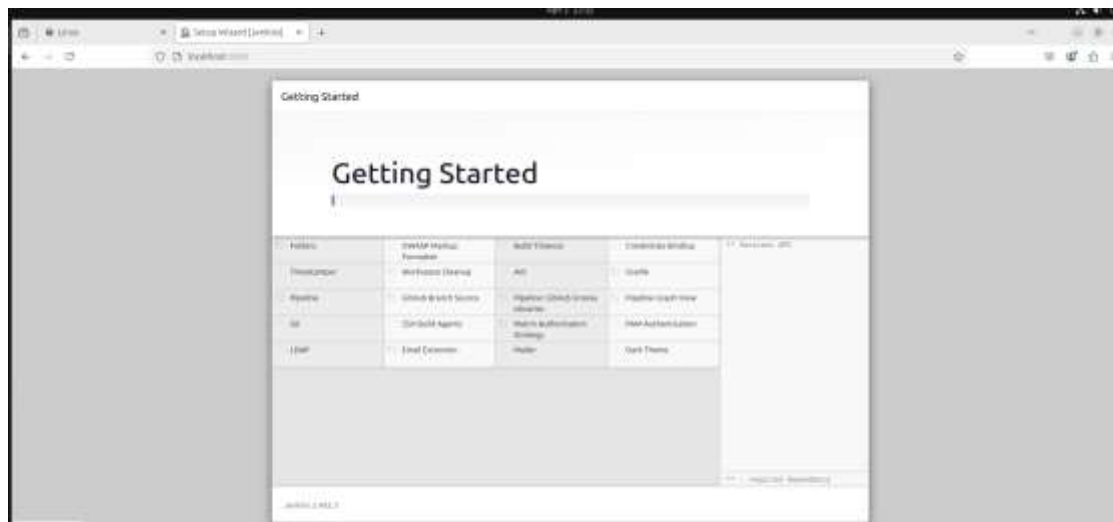
```
ubuntu@ubuntu-Virtual-Platform:~/devops$ sudo more /var/lib/jenkins/secrets/initialAdminPassword
d84680b78b434e5fa5f0077ce5a0ce58
```

**5. Follow the Setup Wizard:**

- Install Suggested Plugins: Click on “Install suggested plugins” for a typical setup.
- Create an Admin User: Follow prompts to create your first admin user.
- Finalize Configuration: Complete the remaining setup steps.



Wait until all the plugins are installed .



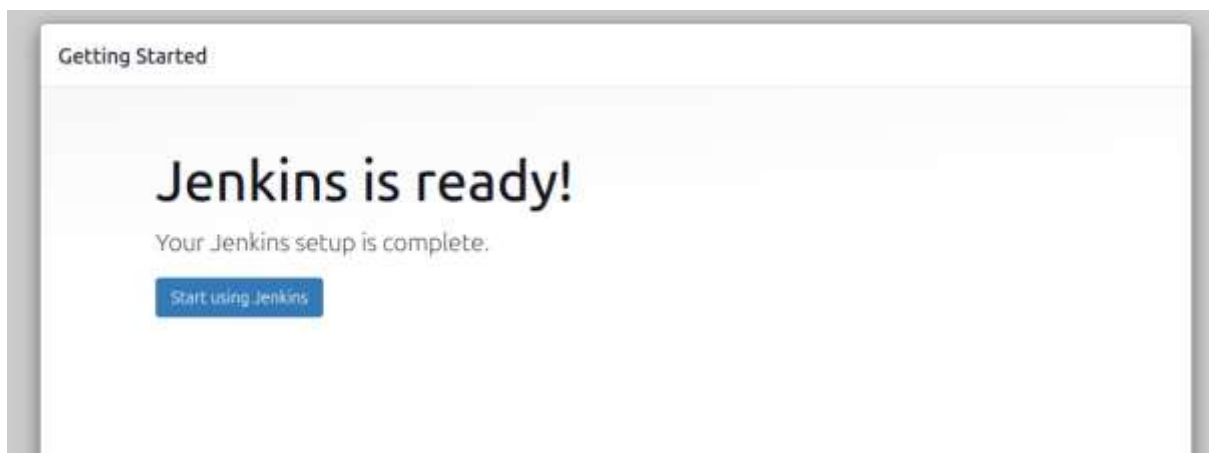
Create your own Jenkins account with all the required details asked .

The image shows the 'Create First Admin User' form in the Jenkins setup wizard. The form is titled 'Create First Admin User' and contains the following fields:

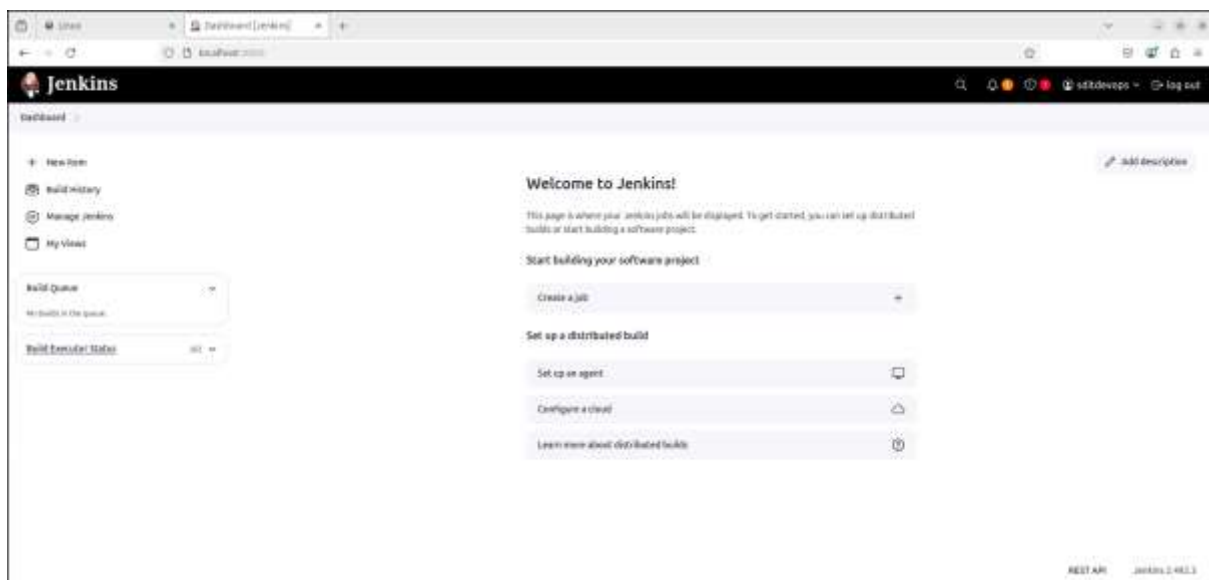
- Username**: A text input field with the value 'devops'.
- Password**: A password input field with the value 'devops'.
- Confirm password**: A password input field with the value 'devops'.
- Full name**: A text input field with the value 'devops'.
- Email address**: A text input field with the value 'devops@gmail.com'.

At the bottom of the form, there are two buttons: 'Skip and continue as admin' and 'Save and Continue'.

After configuring the Jenkins gives the URL:

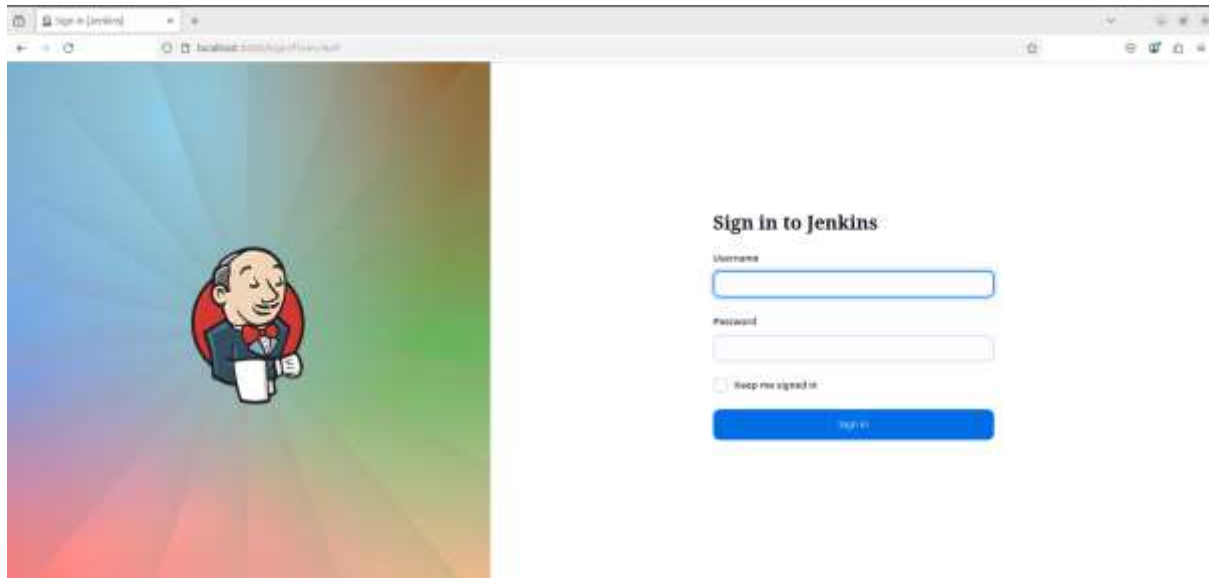


This is the Jenkins Dashboard, which is the main interface of Jenkins after successful installation and login.



To login to your Jenkins account :

- Open web browser
- Navigate to : localhost:8080



Here enter the password and the username that was given when the creation and that will be logged in and you will reach the Jenkins dashboard.

Username:Devops

Password:devops@123

When the account is already created and the other batches has to work the same program so:

sudo apt-get remove purge jenkins

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops$ sudo apt-get remove --purge jenkins
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following packages were automatically installed and are no longer required:
  bnd groovy ivy java-wrappers javascript-common libbcpg-java libbcprov-java libbind-java
  libcommons-codec-java libcommons-collections3-java libcommons-compress-java libcommons-lang-java
  libdd-plist-java libdom4j-java libeclipse-jdt-annotation-java libfelix-framework-java
  libfelix-gogo-runtime-java libfelix-osgi-obr-java libfelix-resolver-java libfindbugs-java
  libgoogle-gson-java libgpars-groovy-java libgradle-core-java libgradle-plugins-java libhttpclient-java
  libhttpcore-java libjarjar-java libjatl-java libjavaewah-java libjaxen-java libjcifs-java
  libjcip-annotations-java libjcsp-java libjformatstring-java libjgit-java libjna-java libjna-jni
  libjs-jquery libjsoup-java libjsr166y-java libkryo-java libkxml2-java libminlog-java
  libmultiverse-core-java libnative-platform-java libnative-platform-jni libnekohtml-java
  libobjenesis-java libosgi-annotation-java libosgi-compendium-java libosgi-core-java
  libplexus-container-default-java libpolyglot-maven-java libqdox-java libreflectasm-java
  libreflectasm-java-doc librhino-java libsimple-http-java libwagon-http-java libxbean-reflect-java
  libxerces2-java libxml-commons-external-java libxpp3-java libxstream-java net-tools python3-netifaces
Use 'sudo apt autoremove' to remove them.
The following packages will be REMOVED:
  jenkins*
0 upgraded, 0 newly installed, 1 to remove and 22 not upgraded.
```

Experiment 6: Continuous Integration with Jenkins: Setting Up a CI Pipeline, Integrating Jenkins with Maven/Gradle, Running Automated Builds and Tests.

What is a CI Pipeline?

A Continuous Integration (CI) Pipeline is a series of automated steps that run every time code is committed to a repository. It helps in automatically building the code, running tests, and integrating changes to catch errors early.

Why Use Jenkins for CI?

Automation: Jenkins automates code building and testing, reducing manual work.

Immediate Feedback: Developers are quickly notified if there are issues.

Extensibility: Jenkins supports many plugins for tools like Git, Maven, and testing frameworks.

Pipeline as Code: Jenkins allows you to define the CI steps in a Jenkinsfile, which can be stored in the same repository as your code.

To install git :

sudo apt update

sudo apt install git -y

```
Get:3 http://in.archive.ubuntu.com/ubuntu oracular-updates/main amd64 git amd64 1:2.45.2-1ubuntu1.1 [4,020 k
B]
Fetched 5,168 kB in 6s (796 kB/s)
Selecting previously unselected package liberror-perl.
(Reading database ... 197734 files and directories currently installed.)
Preparing to unpack .../liberror-perl_0.17029-2_all.deb ...
Unpacking liberror-perl (0.17029-2) ...
Selecting previously unselected package git-man.
Preparing to unpack .../git-man_1%3a2.45.2-1ubuntu1.1_all.deb ...
Unpacking git-man (1:2.45.2-1ubuntu1.1) ...
Selecting previously unselected package git.
Preparing to unpack .../git_1%3a2.45.2-1ubuntu1.1_amd64.deb ...
Unpacking git (1:2.45.2-1ubuntu1.1) ...
Setting up liberror-perl (0.17029-2) ...
Setting up git-man (1:2.45.2-1ubuntu1.1) ...
Setting up git (1:2.45.2-1ubuntu1.1) ...
Processing triggers for man-db (2.12.1-3) ...
```

git -version

```
ubuntu@ubuntu-VMware-Virtual-Platform:~/devops$ git --version
git version 2.45.2
```

1. Setting Up a CI Pipeline with Jenkins (Freestyle Project)

This section explains how to create a CI pipeline as a Freestyle project that integrates with a Maven or Gradle project.

Step 1: Create a New Jenkins Job

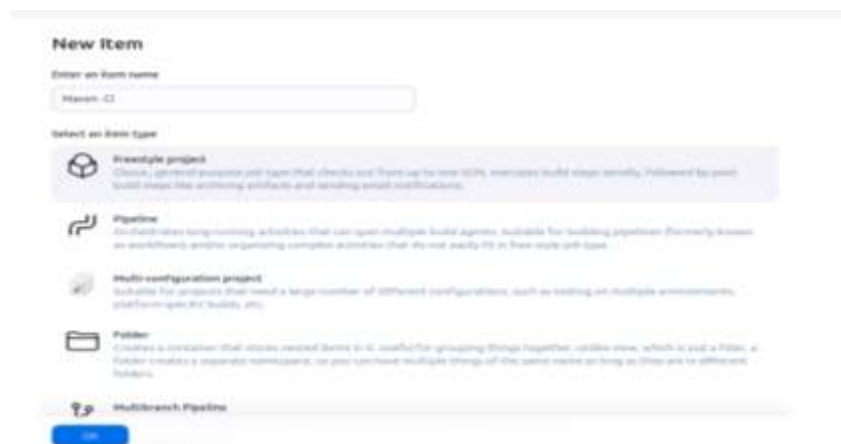
1. Log into Jenkins:

- Open your web browser and navigate to your Jenkins URL (e.g: <http://localhost:8080> or your cloud instance URL).
- Log in with your admin credentials.



2. Create a New Job:

- On the Jenkins dashboard, click on “**New Item**”.
- Enter an Item Name: For example, **Maven-CI** (or Gradle-CI if you prefer Gradle).
- Select “**Freestyle project**”.
- Click “**OK**”.



Step 2: Configure Source Code Management (SCM)

1. Select SCM:

- In the job configuration page, scroll down to the “**Source Code Management**” section.
- Select “**Git**” (if using Git for version control).

2. Enter Repository Details:

- Repository URL: Enter the URL of your Git repository : **https://github.com/spring-projects/spring-petclinic.git**
- Optionally, specify the Branch Specifier (e.g., */main)



Step 3: Add Build Steps

A. For a Maven Project

1. Add Maven Build Step:

- Scroll down to “**Build**” and click on “**Add build step**”.
- Select “**Invoke top-level Maven targets**”.
- Goals: In the Goals field, enter: **clean package**

This command instructs Maven to clean the previous build artifacts, compile the code, run tests, and package the application into a JAR/WAR file.

Build Steps

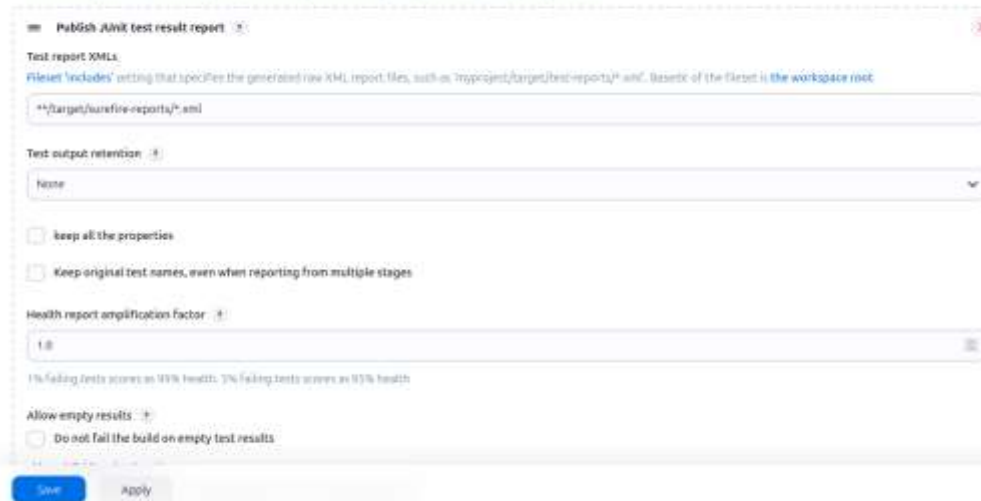
Automate your build process with predefined tasks like code compilation, testing, and deployment.

**Step 4: Configure Post-build Actions****1. Publish Test Results:**

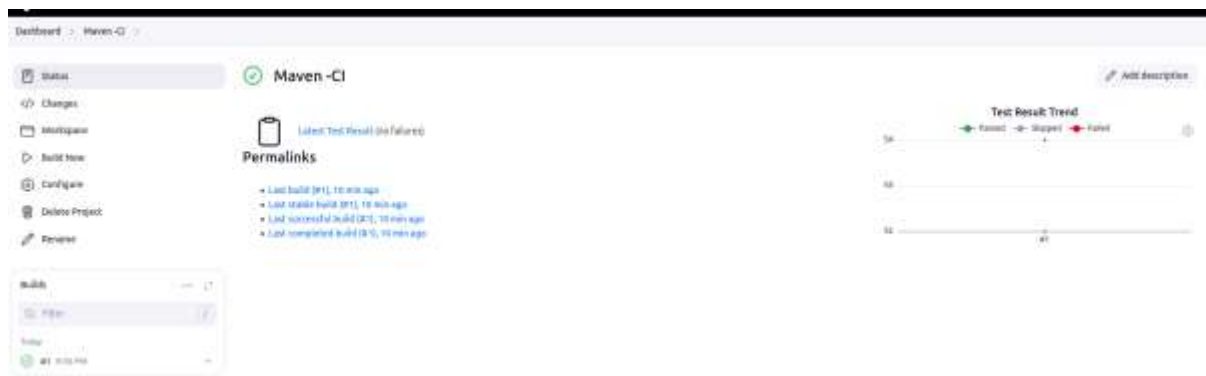
- Scroll down to the **“Post-build Actions”** section.
- Click **“Add post-build action”** and select **“Publish JUnit test result report”**.
- Test Report XMLs: In the field, enter the pattern that matches your test report files. For example: ****/target/surefire-reports/*.xml**

Post-build Actions

Define what happens after a build completes, like sending notifications, archiving artifacts, or triggering other jobs.

**Step 5: Save and Run the Job****1. Save the Configuration:**

- Click **“Save”** at the bottom of the job configuration page.

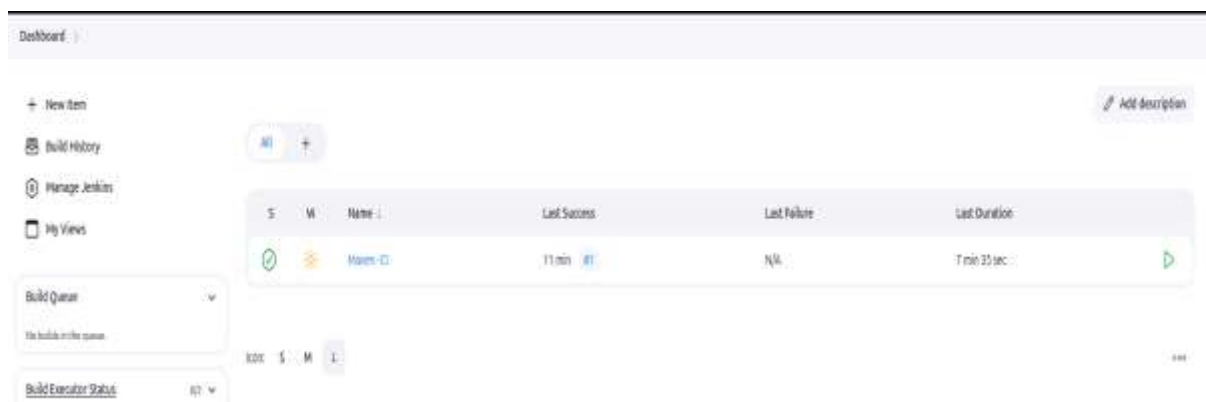


2. Trigger a Build:

- On the job's main page, click **“Build Now”**.
- The build will be added to the build history on the left side.

3. Monitor Build Output:

- Click on the build number (e.g., #1) and then click **“Console Output”**.



- Verify that Jenkins successfully checks out the code, runs the build commands (Maven or Gradle), and executes tests.
- Look for **“BUILD SUCCESS”** or the equivalent output to confirm that the build and tests passed.

```
Dashboard > Myenv > 41 > Console Output

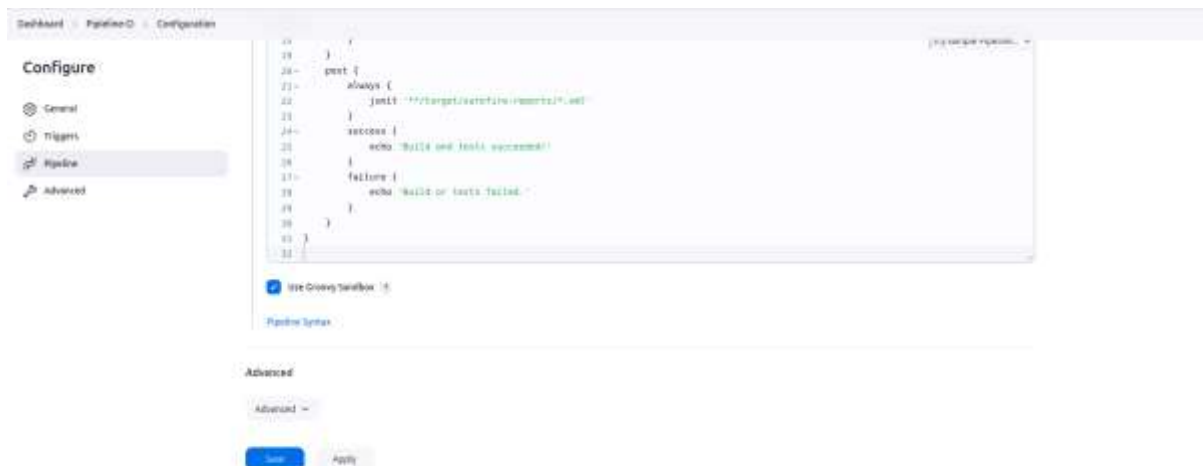
[[1.3442WFO]] [[1.32Tests run: 0, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.139 s -- in org.springframework.samples.petclinic.web.ValidatorTest]]
[[1.3442WFO]]
[[1.3442WFO]] Results:
[[1.3442WFO]]
[[1.3442WFO]] [[1.32Tests run: 54, Failures: 0, Errors: 0, Skipped: 0]]
[[1.3442WFO]]
[[1.3442WFO]] [[1.32maven-plugin:3.4.2:report] [INFO] report on @ [Repackaging-petclinic-3.4.2] --]]
[[1.3442WFO]] [[1.32maven-plugin:3.4.2:report] [INFO] report on @ [Repackaging-petclinic-3.4.2] --]]
[[1.3442WFO]] Loading execution data file /var/lib/jenkins/workspace/Myenv -CI/target/jacoco.exec
[[1.3442WFO]] Analyzed bundle 'petclinic' with 21 classes
[[1.3442WFO]]
[[1.3442WFO]] [[1.32maven-jar-plugin:3.4.2:jar] [INFO] Building jar for [Repackaging-petclinic-3.4.2] --]]
[[1.3442WFO]] Building jar: /var/lib/jenkins/workspace/Myenv -CI/target/spring-petclinic-3.4.2-SNAPSHOT.jar
[[1.3442WFO]]
[[1.3442WFO]] [[1.32spring-boot-maven-plugin:3.4.2:repackage] [INFO] Repackaging the @ [Repackaging-petclinic-3.4.2] --]]
[[1.3442WFO]] Repackaging main artifact /var/lib/jenkins/workspace/Myenv -CI/target/spring-petclinic-3.4.2-SNAPSHOT.jar with repackaged archive, adding nested
dependencies in BOOT-INF/.
[[1.3442WFO]] The original artifact has been renamed to /var/lib/jenkins/workspace/Myenv -CI/target/spring-petclinic-3.4.2-SNAPSHOT.jar.original
[[1.3442WFO]] [[1.32BUILD SUCCESS]]
[[1.3442WFO]]
[[1.3442WFO]] [[1.32BUILD SUCCESS]]
[[1.3442WFO]]
[[1.3442WFO]] Total time: 07:27 min
[[1.3442WFO]] Finished at: 2019-04-21T12:43:28+05:30
[[1.3442WFO]]
[[1.3442WFO]] Recording test results
[[1.3442WFO]] (Checks API) No suitable checks publisher found.
Finished: SUCCESS
```

2. Setting Up a CI Pipeline with Jenkins (Pipeline as Code)

For greater flexibility and version-controlled CI configuration, you can use a Jenkins Pipeline defined in a Jenkinsfile.

Step 1: Create a Pipeline Job

1. Log into Jenkins and click on “**New Item**”.
2. Enter an Item Name: For example, **Pipeline-CI**.
3. Select “**Pipeline**” and click “**OK**”.



New Item

Enter an item name:

Pipeline-CI

Select an item type:



Freestyle project

Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.



Pipeline

Orchestrates long-running activities that can span multiple build agents. Suitable for building pipelines (formerly known as workflows) and/or organizing complex activities that do not easily fit in free-style job type.



Multi-configuration project

Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.



Folder

Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.



Multibranch Pipeline

OK

Experiment 7: Configuration Management with Ansible: Basics of Ansible: Inventory, Playbooks, and Modules, Automating Server Configurations with Playbooks, Hands-On: Writing and Running a Basic Playbook.

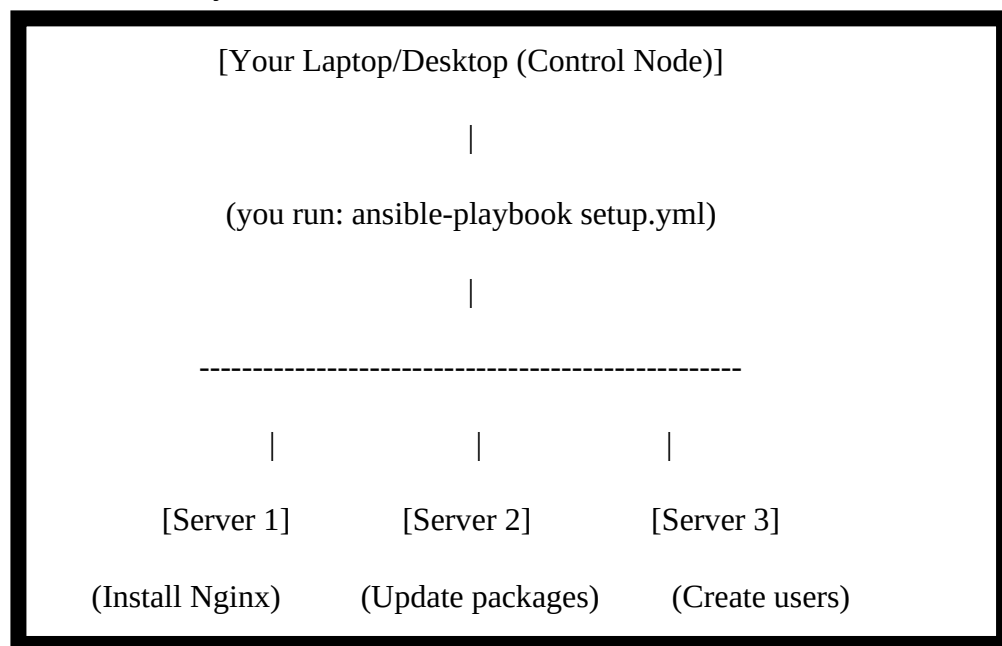
1. Introduction to Ansible

What Is Ansible?

Ansible is a software tool that helps you automate work on computers.

Instead of doing everything manually (like installing software, changing settings, starting services), you write instructions once and Ansible does the work for you automatically on many machines.

Ansible is like a smart robot that can set up and manage computers for you, without you touching each one manually.



2. Installing Ansible on Ubuntu

Before writing a playbook, you need to install Ansible on your control machine (your local Ubuntu system).

Step 1: Update Your System

Open your terminal and run:

sudo apt update

Step 2: Install Ansible

Install Ansible using apt:

sudo apt install ansible -y

```
ubuntu@ubuntu-VHware-Virtual-Platform:~/devops$ sudo apt install ansible -y
The following packages were automatically installed and are no longer required:
  bnd                                libfelix-osgi-obr-java          libjgit-java                   libplexus-container-default-java
  groovy                            libfelix-resolver-java         libjna-java                   libpolyglot-naven-java
  ivy                               libfindbugs-java              libjna-jni                    libqdox-java
  java-wrappers                    libgoogle-gson-java           libjs-jquery                  libreflectasm-java
  javascript-common                libgpars-groovy-java           libjsoup-java                 libreflectasm-java-doc
  libbcpkg-java                   libgradle-core-java           libjsr166y-java              librhino-java
  libbcprov-java                   libgradle-plugins-java        libkryo-java                  libsimple-http-java
  libbindex-java                  libhttpclient-java            libkxml2-java                 libwagon-http-java
  libcommons-codec-java           libhttpcore-java              libminlog-java               libxbean-reflect-java
  libcommons-collections3-java    libjarjar-java                libmultiverse-core-java      libxerces2-java
  libcommons-compress-java        libjatl-java                  libnative-platform-java     libxml-commons-external-java
  libcommons-lang-java            libjavaewah-java              libnative-platform-jni      libxpp3-java
  libdd-plist-java                libjaxen-java                 libnekohtml-java            libxstream-java
  libdom4j-java                   libjcifs-java                  libobjenesis-java           python3-netifaces
  libeclipse-jdt-annotation-java  libjclap-annotations-java     libosgi-annotation-java
  libfelix-framework-java        libjcsp-java                  libosgi-compndium-java
  libfelix-gogo-runtime-java      libjformatstring-java         libosgi-core-java

Use 'sudo apt autoremove' to remove them.

Installing:
  ansible

Suggested packages:
  cowsay sshpass
```

Verify the installation by checking the version:

ansible --version

```
ubuntu@ubuntu-VHware-Virtual-Platform:~/devops$ ansible --version
ansible [core 2.16.3]
  config file = None
  configured module search path = ['/home/ubuntu/.ansible/plugins/modules', '/usr/share/ansible/plugins/modules']
  ansible python module location = /usr/lib/python3/dist-packages/ansible
  ansible collection location = /home/ubuntu/.ansible/collections:/usr/share/ansible/collections
  executable location = /usr/bin/ansible
  python version = 3.12.7 (main, Feb  4 2025, 14:46:03) [GCC 14.2.0] (/usr/bin/python3)
  jinja version = 3.1.3
  libyaml = True
```

3. Creating an Ansible Inventory

An inventory file lists the hosts you want to manage. For this experiment, you can use the local host.

Step 1: Create an Inventory File

1. Open your text editor to create a file called hosts.ini:
2. nano hosts.ini

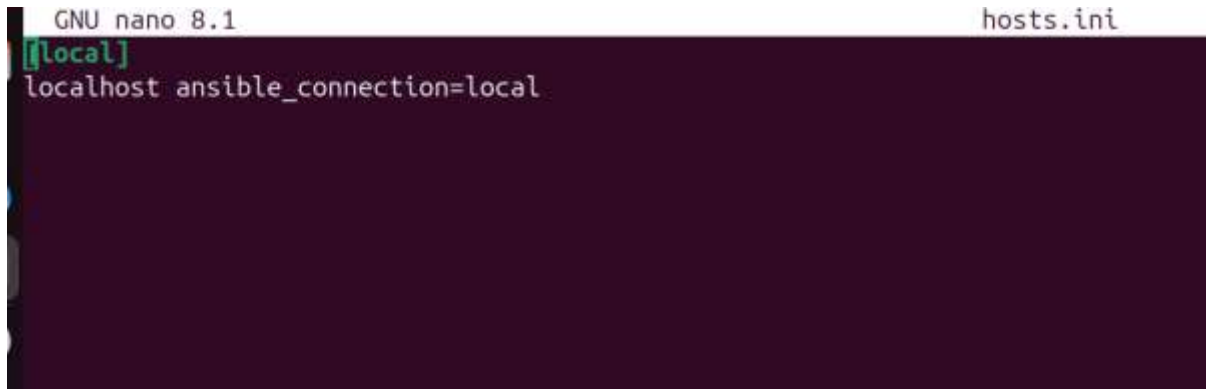
3. Add the following content to define the local host:

```
[local]
```

```
localhost ansible_connection=local
```

Explanation:

- [local] is a group name.
- localhost is the target host.
- ansible_connection=local tells Ansible to execute commands on the local machine without SSH.



```
GNU nano 8.1 hosts.ini
[local]
localhost ansible_connection=local
```

4. Writing a Basic Ansible Playbook

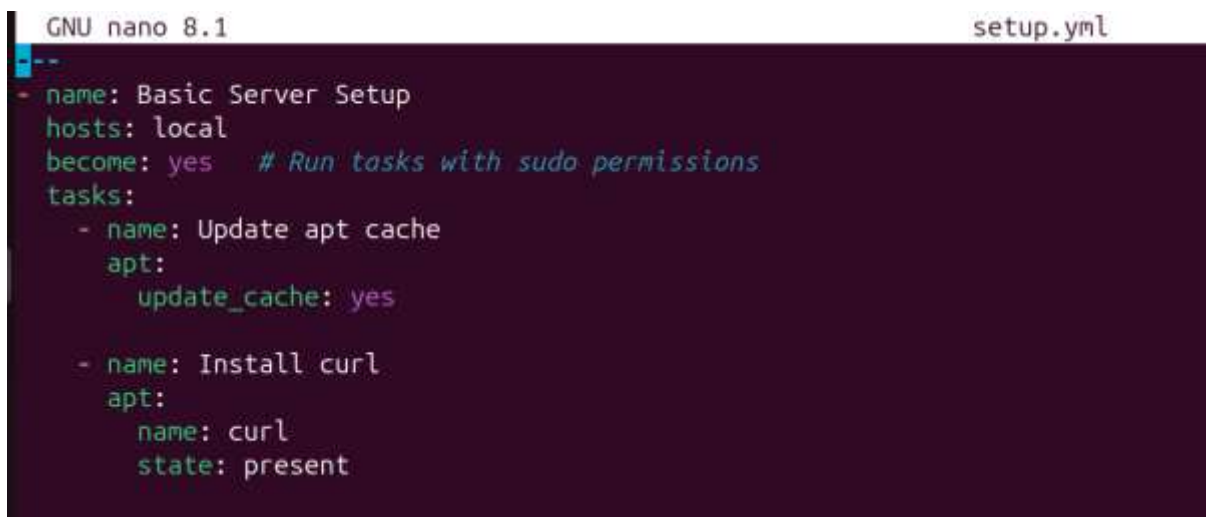
You will now create a simple playbook that performs two common tasks:

- Updating the apt cache
- Installing a package (e.g., curl)

Step 1: Create the Playbook File

1. Open your text editor to create a file called setup.yml:
2. **nano setup.yml**

```
---  
  
- name: Basic Server Setup  
  
  hosts: local  
  
  become: yes  # Run tasks with sudo permissions  
  
  tasks:  
  
    - name: Update apt cache  
  
      apt:  
  
        update_cache: yes  
  
  
    - name: Install curl  
  
      apt:  
  
        name: curl  
  
        state: present
```

A screenshot of a terminal window with a dark purple background. The title bar at the top shows 'GNU nano 8.1' on the left and 'setup.yml' on the right. The terminal content displays the same Ansible playbook code as the previous block, with syntax highlighting: green for keywords like 'name', 'hosts', 'tasks', 'apt', and 'state'; blue for comments like '# Run tasks with sudo permissions'; and purple for variable names like 'update_cache' and 'name' within the 'apt' block. The code is indented to show the hierarchical structure of the playbook.

```
GNU nano 8.1                                     setup.yml  
---  
- name: Basic Server Setup  
  hosts: local  
  become: yes  # Run tasks with sudo permissions  
  tasks:  
    - name: Update apt cache  
      apt:  
        update_cache: yes  
  
    - name: Install curl  
      apt:  
        name: curl  
        state: present
```

5. Running the Ansible Playbook

Step 1: Execute the Playbook

In your terminal, run the following command:

ansible-playbook -i hosts.ini setup.yml --ask-become-pass

```
ubuntu@ubuntu-Virtual-Platform:~/devops$ nano hosts.ini
ubuntu@ubuntu-Virtual-Platform:~/devops$ nano setup.yml
ubuntu@ubuntu-Virtual-Platform:~/devops$ ansible-playbook -i hosts.ini setup.yml --ask-become-pass
BECOME password:

PLAY [Basic Server Setup] *****

TASK [Gathering Facts] *****
ok: [localhost]

TASK [Update apt cache] *****
changed: [localhost]

TASK [Install curl] *****
ok: [localhost]

PLAY RECAP *****
localhost                : ok=3    changed=1    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

ubuntu@ubuntu-Virtual-Platform:~/devops$
```

To remove ansible and re install that :

sudo apt remove ansible

Experiment 8: Practical Exercise: Set Up a Jenkins CI Pipeline for a Maven Project, Use Ansible to Deploy Artifacts Generated by Jenkins.

Overview

In this experiment, you will:

Set up a Jenkins job to automatically build a Maven project from source control.

Archive the build artifact (a JAR file) produced by Maven.

Integrate an Ansible deployment step within Jenkins (using a post-build action) to deploy the artifact to a target location. Verify that the artifact is deployed successfully.

This exercise demonstrates how Continuous Integration (CI) and automated configurationmanagement can work together to streamline the build-and-deploy process.

Step 1: start your Jenkins account.

```
sudo systemctl start jenkins.
```

```
sudo systemctl status jenkins.
```

Step 2: Preparing the Maven Project

Go to Jenkins → New Item

Name: Maven-CI-Pipeline

Type: Freestyle project → Click OK

Step 3: Configure the Job

Source Code Management → Git

<https://github.com/spring-projects/spring-petclinic.git>

Branch specifier: */main

Step 4: Build Step → Add → Invoke top-level Maven targets
clean install

Step 5: Build Step → Execute Shell

```
ansible-playbook /var/lib/jenkins/workspace/spring-petclinic-pipeline/deploy.yml -i  
/home/ubuntu/hosts.ini -l remote
```

Build Steps

Automate your build process with ordered tasks like code compilation, testing, and deployment.

Invoke top-level Maven targets

Maven Version

(Default)

Goals

clean install

Advanced

Execute shell

Command

See the list of available environment variables

```
ansible-playbook /var/lib/jenkins/workspace/spring-petclinic-pipeline/deploy.yml -i /home/ubuntu/hosts.ini -l remote
```

Step 6: Post-build Actions → Add → Archive the artifacts

target/*.jar

click on Save

Step 7: Open terminal new terminal :

mkdir myapp

cd myapp

nano pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
```

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
```

```
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
```

```
http://maven.apache.org/xsd/maven-4.0.0.xsd">
```

```
<modelVersion>4.0.0</modelVersion>
```

```
<groupId>com.example</groupId>
```

```
<artifactId>myapp</artifactId>
```

```
<version>1.0-SNAPSHOT</version>
```

```
<packaging>jar</packaging>
```

```
<build>
```

```
<plugins>
```

```
<plugin>
```

```
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.8.1</version>
<configuration>
  <source>11</source>
  <target>11</target>
</configuration>
</plugin>
</plugins>
</build>
</project>
```

```
mkdir -p src/main/java/com/example
gedit src/main/java/com/example/App.java package com.example;
public class App
{
    public static void main(String[] args) {
        System.out.println("Hello from Jenkins Maven CI!");
    }
}
mvn clean install
```

```
[INFO] --- maven-compiler-plugin:3.8.1:testCompile (default-testCompile) @ myapp ---
[INFO] No sources to compile
[INFO] --- maven-surefire-plugin:2.12.4:test (default-test) @ myapp ---
[INFO] No tests to run.
[INFO] --- maven-jar-plugin:2.4:jar (default-jar) @ myapp ---
[INFO] Building jar: /home/ubuntu/devops/myapp/target/myapp-1.0-SNAPSHOT.jar
[INFO] --- maven-install-plugin:2.4:install (default-install) @ myapp ---
[INFO] Installing /home/ubuntu/devops/myapp/target/myapp-1.0-SNAPSHOT.jar to /home/ubuntu/.m2/repository/com/example/myapp-1.0-SNAPSHOT.jar
[INFO] Installing /home/ubuntu/devops/myapp/pom.xml to /home/ubuntu/.m2/repository/com/example/myapp-1.0-SNAPSHOT.pom
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 4.818 s
[INFO] Finished at: 2025-05-05T21:38:00+05:30
[INFO] -----
```

9. Step 8: Open terminal where you had started jenkins :

nano /home/ubuntu/hosts.ini

[remote]

192.168.1.100 ansible_user=ubuntu ansible_ssh_private_key_file=/home/ubuntu/.ssh/id_rsa

nano /home/ubuntu/deploy.yml

```
GNU nano 8.1 /home/ubuntu/deploy.yml
--
- name: Deploy Spring PetClinic
  hosts: remote
  tasks:
    - name: Copy the jar file to remote server
      copy:
        src: /var/lib/jenkins/workspace/Maven-CI-Pipeline/target/spring-petclinic-*.jar
        dest: /home/ubuntu/petclinic.jar
        mode: '0755'

    - name: Run the Spring PetClinic app
      shell: nohup java -jar /home/ubuntu/petclinic.jar > /home/ubuntu/output.log 2>&1 &
      async: 30
      poll: 0
```

sudo mkdir -p /var/lib/jenkins/workspace/spring-petclinic-pipeline/

sudo mv /home/ubuntu/deploy.yml /var/lib/jenkins/workspace/spring-petclinicpipeline/deploy.yml

10. Step 9: Go to your jenkins dashboard :

- click on > Build Now • console output.

```
Dashboard > MavenCI-Pipeline > #1 > Console Output

[11:34:03][n] Installing /var/lib/jenkins/workspace/MavenCI-Pipeline/pom.xml to /var/lib/jenkins/.m2/repository/org/springframework/samples/spring-petclinic/1.4.0-SNAPSHOT/spring-petclinic-1.4.0-SNAPSHOT.jar
[11:34:03][n] Installing /var/lib/jenkins/workspace/MavenCI-Pipeline/target/spring-petclinic-1.4.0-SNAPSHOT.jar to /var/lib/jenkins/.m2/repository/org/springframework/samples/spring-petclinic/1.4.0-SNAPSHOT/spring-petclinic-1.4.0-SNAPSHOT.jar
[11:34:03][n] Installing /var/lib/jenkins/workspace/MavenCI-Pipeline/target/classes/META-INF/about/application.cdx.jar to /var/lib/jenkins/.m2/repository/org/springframework/samples/spring-petclinic/1.4.0-SNAPSHOT/spring-petclinic-1.4.0-SNAPSHOT-cdx.jar
[11:34:03][n] [n]-----[n]
[11:34:03][n] [11:34:03][n] SUCCESS
[11:34:03][n] [n]-----[n]
[11:34:03][n] Total time: 02:16 sjs
[11:34:03][n] Finished at: 2025-05-03T12:54:45-05:30
[11:34:03][n] [n]-----[n]
[MavenCI-Pipeline] 4,3kx/vb -vs /tmp/jenkins1374266/4102823895.sh
^ ssh(16-plugbook /var/lib/jenkins/workspace/spring-petclinic/pipeline/overlay.yml -f /home/ubuntu/hosts.ini -f remote
[WARNING]: Unable to parse /home/ubuntu/hosts.ini as an inventory source
[WARNING]: No inventory was parsed, only implicit localhost is available
[WARNING]: provided hosts list is empty, only localhost is available. Note that
the implicit localhost does not match 'all'
[WARNING]: Could not match supplied host pattern, ignoring: remote

PLAY [Deploy Spring Petclinic] *****
skipping: no hosts matched

PLAY RECAP *****

Annoying artifacts
Finished: SUCCESS
```